



Progress on Defect Predictor

By: Huy Nguyen and Edward Feng



Research DefectPredictorTemplate

- Tested harness.py
- Researched precision, recall, and f1 score
- Referenced the TA template
 - Get commits

Revisit Potential Parameters in the Formula

- Recency (Date committed)
- Frequency (How often the files are changed)
- Message keywords
 - Very depending on developer preference so might not work/less weight
- Number of lines changed
- How many lines of code in the file

get_fixing_commits()

- Referenced repominer
- FixingCommitClassifier class
 - fixes_syntax
 - Has_defect_pattern
 - Checks the commit message against a list of keywords

```
def get_fixing_commits(self, num_workers=8) -> Dict[str, List[str]]:
    """
    Return a list of bug-fixing commit hash, categorized as fixing "conditionals", "configuration data",
    "dependencies", "documentation", "idempotency", "security", "service", "syntax".

    This method returns the commits whose message indicates defective scripts.
    'Note:' Beside returning the list of bug-fixing commits, it also updates the attribute ``fixing_commits``.

    Reference: https://github.com/radon-h2020/radon-repository-miner/tree/master

    Parameters
    -----
    num_workers : int
        Number of threads. Default 8.

    Returns
    -----
    List[str]
        A dictionary of bug-fixing commits hashes and boolean values for every fixing labels.
        {'hash1': ['SERVICE', 'SYNTAX', ...]}
    """
    commits_labels = {}
    commits = []

    for commit in Repository(self.path_to_repo, only_in_branch=self.branch, num_workers=num_workers).traverse_commits():
        if commit.hash in self.fixing_commits:
            continue

        fcc = self.FixingCommitClassifier(commit)

        if fcc.fixes_syntax():
            commits_labels.setdefault(commit.hash, []).append('SYNTAX')

        if commit.hash in commits_labels:
            commits.append(commit.hash)
```

```
def has_defect_pattern(self, text: str) -> bool:
    """
    Check if the given text contains any defect-related patterns.

    Parameters
    -----
    text: str
        The text to check for defect patterns.

    Returns
    -----
    bool
        True if defect-related patterns are found, False otherwise.
    """
    string_pattern = ['error', 'bug', 'fix', 'issue', 'mistake', 'incorrect', 'fault', 'defect', 'flaw', 'update', 'modified']
    return any(word in text.lower() for word in string_pattern)
```

Testing

- Created test_functions.py
- Test functions calculate_recency() and calculate_frequency using the sample repo

```
string_pattern = ['error', 'bug', 'fix', 'issue', 'mistake', 'incorrect', 'fault', 'defect', 'flaw', 'update', 'modified']
```

```
edwardfeng@Edwards-MacBook-Air-2 csc495-project % docker run --rm -it bug_detection  
Retrieving modified Python files...  
Modified files debug output: {'word_frequency.py', 'buggy_script.py'}
```

```
Calculating recency scores...
```

```
Recency Scores (File Name -> Recency Value):
```

```
buggy_script.py -> 2025-02-28 19:53:53-05:00
```

```
word_frequency.py -> 2025-03-02 18:05:42-05:00
```

```
Calculating frequency scores...
```

```
Frequency Scores (File Name -> Frequency Value):
```

```
buggy_script.py -> 2
```

```
word_frequency.py -> 3
```

Commits on Mar 2, 2025

Update word_frequency.py

 manideepika21 authored 3 weeks ago

Update word_frequency.py

 manideepika21 authored 3 weeks ago

Commits on Feb 28, 2025

Update word_frequency.py

 manideepika21 authored 3 weeks ago

Create word_frequency.py

 manideepika21 authored 3 weeks ago

Create factorial.py

 manideepika21 authored 3 weeks ago

Update buggy_script.py

 manideepika21 authored 3 weeks ago

modified buggy_script.py

 manideepika21 authored 3 weeks ago

Created buggy_script.py

 manideepika21 authored 3 weeks ago

Build+Testing Demo

Huy

Important Finds

- The “to_date” in the config file is up to but not inclusive
 - 2025-03-02 does not include the commits on March 2nd
 - We had to change it to 2025-03-03

What's next?

- Entering Sprint 3/Iteration 2
 - Collect feedback and continue implementation
 - Test calculate risk using the scores we currently have
 - Implement normalizer
 - Wait for harness update to test result